MACHINE LEARNING · PBL-II · PROJECT REVIEW 2

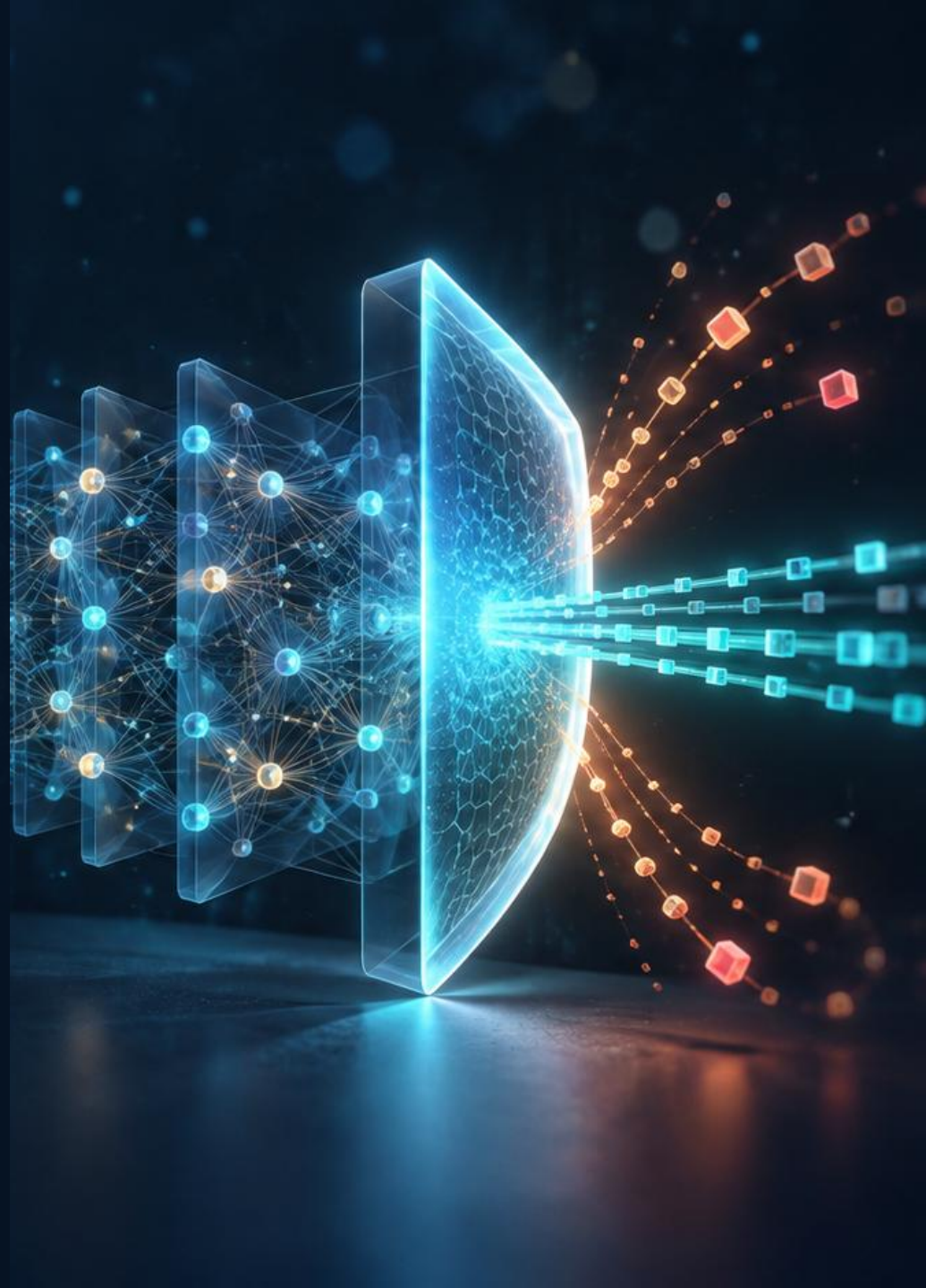
Jailbreaker-LLM

Finding LLM compliance states without labels: PCA, K-means, Gaussian mixtures with EM, and hierarchical clustering

Unit 3: mixture models, EM, latent dimensions and clustering
NumPy analysis plus standalone K-means with no imports

Ananya Kar

RA2411003010453 · 21CSC305P



INTERACTIVE CONTENTS

Choose a section

Each card jumps to its section. Each model is shown as its mathematical model, then its NumPy code, then the measured result.

01 **Problem and Review 2 scope**
OPEN SECTION · SLIDE 03

02 **Dataset and setup**
OPEN SECTION · SLIDE 05

03 **PCA**
OPEN SECTION · SLIDE 07

04 **K-means and GMM-EM**
OPEN SECTION · SLIDE 08

05 **Hierarchical and results**
OPEN SECTION · SLIDE 10

06 **Code, limits, references**
OPEN SECTION · SLIDE 12



What Review 2 adds to the project

The GitHub codebase adds unsupervised models and a no-import K-means script

REVIEW 2 BRIEF

Draft the mathematical model for the problem and implement it as a Python script, avoiding Python libraries.

Component	Marks
Mathematical modelling of the algorithm	2
Python script for the mathematical model	3

- NumPy models: src/analysis/unsupervised.py
- No-import script: review2_kmeans_pure_python.py
- Both fit synthetic judge scores without labels

HOW THE REVIEWS BUILD ON EACH OTHER

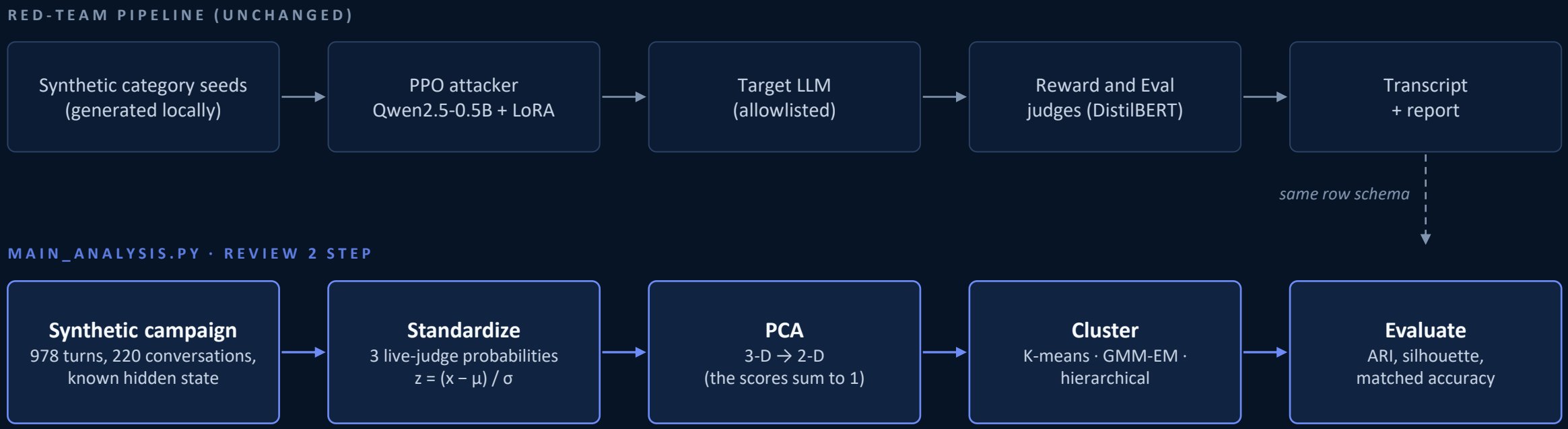
Review	Content	Status
1	EDA, linear regression, logistic regression and SVM classification	Presented
2	PCA, K-means, Gaussian mixture (EM), hierarchical clustering	This deck
3	HMM, CART and ensembles, comparative analysis	Next

Same question as Review 1, now without labels: can the hidden compliance state be recovered from judge scores alone?

The labels are used only afterwards, to score the clusters.

Where the unsupervised step sits in the red-team system

Judges score every turn of a red-team conversation. Unsupervised models test whether those scores alone reveal the target's state



QUESTION

Without labels, do judge scores fall into three groups that match refusal, partial compliance and full violation?

WHY IT MATTERS

Human labels are expensive. If score clusters track the true state, clusters can flag turns for review without a labelled set.

A simulated campaign whose hidden states are known

The repository generates simulated conversations with known states so each cluster can be checked

SYNTHETIC MULTI-TURN CAMPAIGN

978	220	3	6
turns	conversations	target profiles	max turns

Hidden state z	Turns
Refusal	523
Partial compliance	311
Full violation	144

Rows use the real campaign transcript schema (data/generate_synthetic_campaign.py, seed 42), so the same code runs on a real campaign.

GENERATING MODEL

$z_1 \sim \mathrm{Cat}(0.75, 0.20, 0.05)$

$z_t \mid z_{t-1} \sim A_{target}[z_{t-1}, \cdot]$

$(p_{ref}, p_{par}, p_{full}) \sim \mathrm{Dirichlet}(\alpha_{z_t})$

A conversation stops when the judge's P(full) ≥ 0.5, or after 6 turns

DATA BOUNDARY

These results use the generated synthetic campaign. Review 1 examined a separate real-data baseline; do not compare the two as one dataset.

Features, standardization and the simplex constraint

Turn-level models use the three judge probabilities; hierarchical clustering uses one row per conversation

TURN-LEVEL INPUT

$x_n = (P_refusal, P_partial, P_full) \in \mathbb{R}^3$

$z_n = (x_n - \mu) / \sigma$ (per column)

P_refusal + P_partial + P_full = 1

Because of this constraint the points lie on a plane, so one direction carries no variance. PCA should find it.

CONVERSATION-LEVEL INPUT (HIERARCHICAL)

Mean P_refusal, P_partial and P_full; peak P_full; number of turns; final P_full. That is 220 × 6, standardized.

CODE SNIPPET — src/analysis/unsupervised.py

```
01  df = load_transcript_rows().dropna(subset=['true_compliance_state'])
02  y = df['true_compliance_state'].map(COMPLIANCE_STATES.index).values
03  X = df[SCORE_COLUMNS].values
04  X = (X - X.mean(axis=0)) / X.std(axis=0)
05
06  mean, U, ratio = pca(X)
07  Z = (X - mean) @ U[:, :2]      # 2-D principal subspace
08
09  km_labels = kmeans(Z, 3)[0]
10  gmm_labels = gmm_em(Z, 3)[0]
11  hc_labels = agglomerative(C, 3, 'complete')
```

y is never passed to a model. It is only compared with the clusters afterwards.

PCA shows three judge scores carry only two dimensions

Eigen-decomposition of the covariance; the third eigenvalue is exactly zero because the probabilities sum to 1

MATHEMATICAL MODEL

$$S = (1/N)\,\Sigma_n\,(x_n - \bar{x})(x_n - \bar{x})^T$$

$$\mathbf{S\,u_i = \lambda_i\,u_i,\quad \lambda_1 \geq \lambda_2 \geq \lambda_3}$$

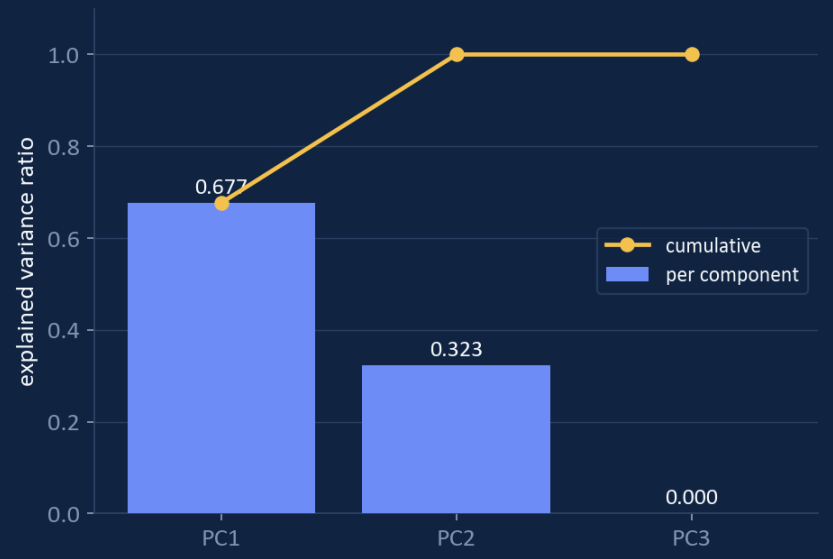
$$z_n = U_M^T\,(x_n - \bar{x})$$

$$\text{ratio}_i = \lambda_i\,/\,\Sigma_j\,\lambda_j$$

Keep M = 2. Keeping the zero-variance direction would make every Gaussian covariance in the GMM singular.

CODE

```
01  S = (X - mean).T @ (X - mean) / len(X)
02  eigvals, eigvecs = np.linalg.eigh(S)
03  order = np.argsort(eigvals)[::-1]
```



67.7 %
PC1: refusal versus the rest

32.3 %
PC2: partial versus full

0.000
PC3: the simplex constraint

Keep 2 PCs: 100 % of the variance

K-means alternates nearest-centre assignment and mean updates

k-means++ seeding with 10 restarts; k is chosen with the elbow in distortion and with the silhouette

MATHEMATICAL MODEL

$J = \Sigma_n \Sigma_k r_{nk} \|x_n - \mu_k\|^2$

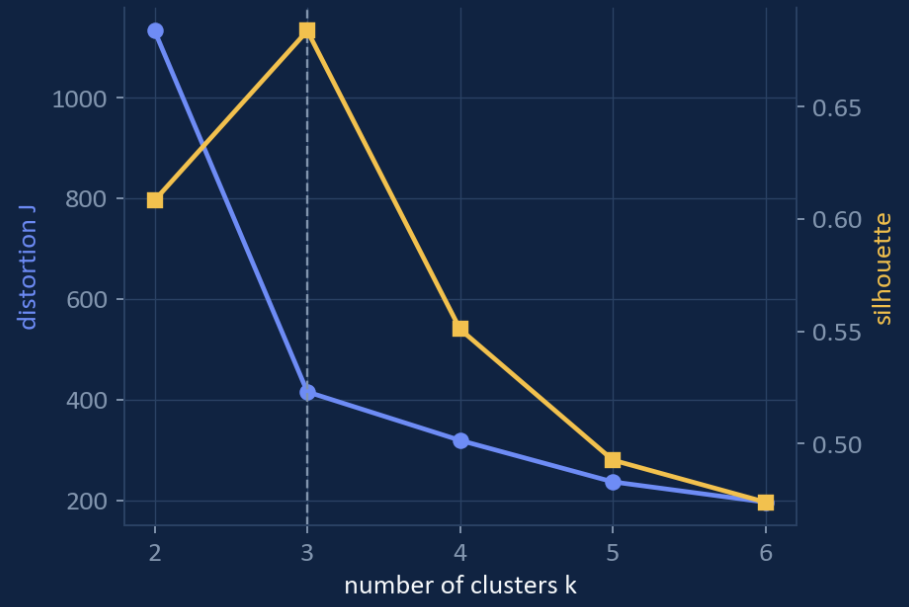
E-step: $r_{nk} = 1$ if $k = \text{argmin}_j \|x_n - \mu_j\|^2$

M-step: $\mu_k = \Sigma_n r_{nk} x_n / \Sigma_n r_{nk}$

k-means++: $P(\text{next centre} = x_n) \propto \min_j \|x_n - \mu_j\|^2$

CODE SNIPPET — unsupervised.py::kmeans

```
# E-step: nearest centre
labels = sq_dists(X, mu).argmin(axis=1)
# M-step: mean of each cluster
new_mu = np.array([X[labels == j].mean(axis=0)
                   for j in range(k)])
```



k = 3
elbow + peak silhouette

0.684
silhouette at k = 3

0.924
ARI vs true state

Gaussian mixture fitted with expectation–maximization

Soft responsibilities instead of hard assignments; initialized from K-means, converged in 41 iterations

MATHEMATICAL MODEL

$$p(x) = \Sigma_k \pi_k \, N(x \mid \mu_k, \Sigma_k)$$

E: $\gamma_{nk} = \pi_k \, N(x_n \mid \mu_k, \Sigma_k) \,/\, \Sigma_j \, \pi_j \, N(x_n \mid \mu_j, \Sigma_j)$

M: $N_k = \Sigma_n \gamma_{nk}$, $\pi_k = N_k / N$, $\mu_k = \Sigma_n \gamma_{nk} x_n / N_k$

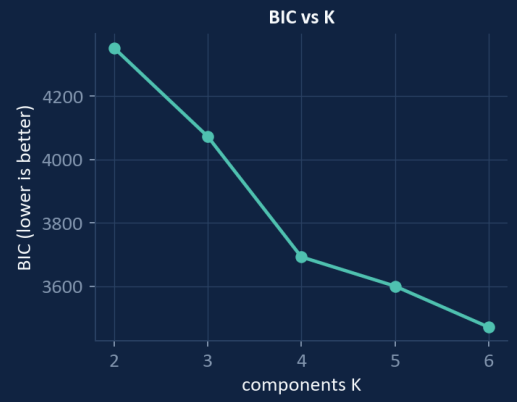
$$\Sigma_k = \Sigma_n \gamma_{nk} (x_n - \mu_k)(x_n - \mu_k)^T / N_k$$

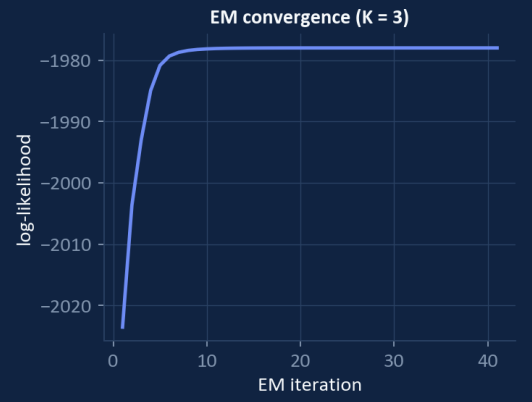
$\ln p(X) = \Sigma_n \ln \Sigma_k \pi_k \, N(x_n \mid \mu_k, \Sigma_k)$ (never decreases)

$BIC = -2 \ln L + p \ln N$

E-STEP — unsupervised.py::gmm_em

```
lj = np.column_stack([
    np.log(pi[j]) + log_gauss(X, mu[j], cov[j])
    for j in range(k)])
m = lj.max(axis=1, keepdims=True)          # log-sum-exp
log_norm = m[:, 0] + np.log(np.exp(lj - m).sum(axis=1))
R = np.exp(lj - log_norm[:, None])         # gamma_nk
```





0.865	0.951	41
ARI vs true state	accuracy after matching	EM iterations

BIC keeps falling as K grows: the Dirichlet-shaped clusters are skewed, so extra Gaussians help fit them. The silhouette still picks 3.

Agglomerative clustering of whole conversations

220 conversations × 6 features; one merge loop covers all four linkages through the Lance–Williams update

LANCE–WILLIAMS UPDATES

single $d(AB, C) = \min(d_{AC}, d_{BC})$

complete $d(AB, C) = \max(d_{AC}, d_{BC})$

average $(n_A d_{AC} + n_B d_{BC}) / (n_A + n_B)$

Ward $d^2 = ((n_A+n_C)d^2_{AC} + (n_B+n_C)d^2_{BC} - n_C d^2_{AB}) / (n_A+n_B+n_C)$

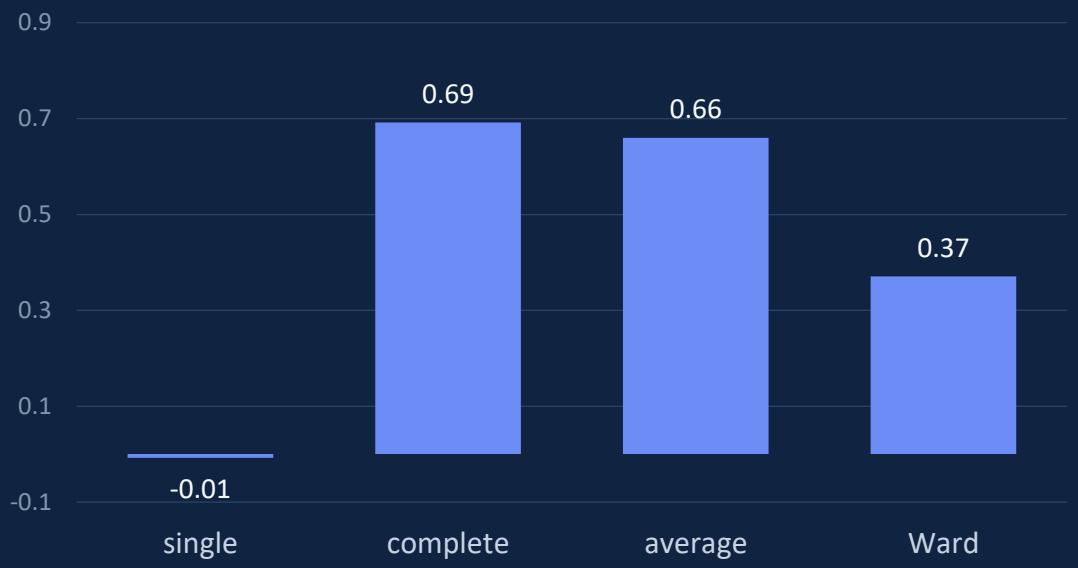
MEASURING DISSIMILARITY

silhouette $s = (b - a) / \max(a, b)$

ARI $= (\Sigma\, C(n_{ij},2) - E) / (\tfrac{1}{2}(\Sigma\, C(a_i,2) + \Sigma\, C(b_j,2)) - E)$

matched accuracy = max over label permutations

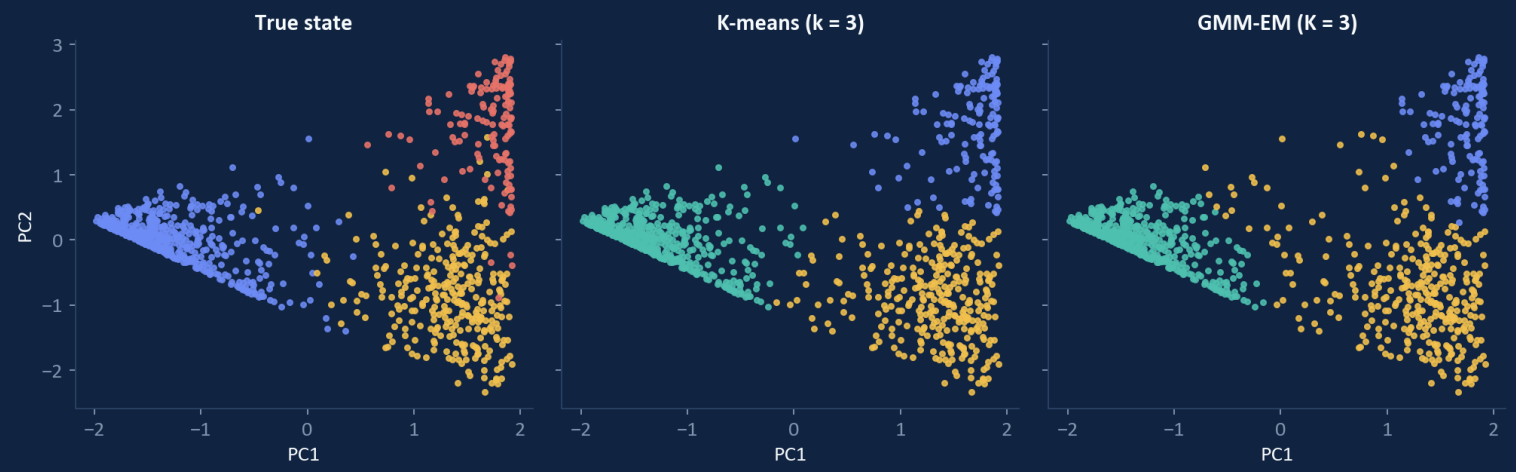
ARI VS FINAL CONVERSATION STATE, BY LINKAGE



Single linkage chains every point into one cluster. Complete linkage separates the outcome groups best, so it is the reported model.

Without labels, K-means recovers 97 % of the hidden states

Clusters in the 2-D principal subspace, compared with the true state afterwards



Model	Implementation	ARI	Matched acc.	Silhouette
K-means (k = 3)	NumPy (Review 2)	0.924	0.969	0.684
GMM-EM (K = 3)	NumPy (Review 2)	0.865	0.951	0.652
Hierarchical, complete (conversations)	NumPy (Review 2)	0.692	0.805	0.423
K-means	scikit-learn baseline	0.924	–	0.684
DBSCAN	scikit-learn baseline	0.883	–	0.725

- The NumPy K-means matches scikit-learn exactly (ARI 0.924).
- Most errors fall on the partial / full boundary, where the judge scores overlap.

Review 2 code and commands

Step 6 runs the models; a standalone K-means script uses no imports

Function in src/analysis/unsupervised.py	Model
pca(X)	Covariance eigen-decomposition, explained-variance ratio
kmeans(X, k)	k-means++ seeding, Lloyd iterations, best of 10
gmm_em(X, k)	EM with log-sum-exp, log-likelihood trace, BIC
agglomerative(X, k, linkage)	Lance–Williams: single / complete / average / Ward
silhouette, adjusted_rand, matched_accuracy	Cluster evaluation
main()	Runs everything, writes metrics.json and 2 plots

Model fitting uses no ready-made ML algorithm.

COMMANDS

```
01  python main_analysis.py
02  python review2_kmeans_pure_python.py
03  python -m pytest tests/test_unsupervised.py
```

VERIFICATION

3 / 3

- Focused Review 2 tests pass
- No-import K-means: 96.9% matched accuracy on synthetic rows
- EM log-likelihood never decreases
- PCA finds the zero-variance simplex direction

What the results do not show yet, and the Review 3 plan

Review 3 compares these models with Unit 4 and 5 algorithms

LIMITATIONS

- The data is simulated. High scores show the method recovers a known structure, not real-world performance
- Turn-level clustering ignores turn order, so each turn is treated as independent
- The Gaussian assumption is wrong for Dirichlet-shaped scores, so BIC keeps preferring more components
- Hierarchical clustering is $O(N^2)$ in memory, which is fine for 220 conversations but not for large campaigns

REVIEW 3 · UNITS 4 AND 5

Model	Use on this data
HMM: forward–backward, Viterbi	Add turn order: decode the state from score sequences
Baum–Welch	Learn the transition matrix and compare it with the true A
CART	Interpretable rules over judge scores
Boosting, bagging	Compare ensembles with the clusters and the Review 1 classifiers

Preview: the existing hmmlearn HMM already reaches ARI 0.868 with matched accuracy 0.951. That is how much turn order adds.

Textbooks, data and method sources

Equations follow the course reference books; section numbers are given in the speaker notes

1. Bishop, Pattern Recognition and Machine Learning
Springer, 2006 · ch. 9 (K-means, mixtures, EM)

2. Murphy, Machine Learning: a Probabilistic Perspective
MIT Press, 2012 · ch. 12 (PCA), 25 (clustering)

3. Arthur and Vassilvitskii, k-means++
SODA 2007

4. Chao et al., JailbreakBench
NeurIPS Datasets and Benchmarks 2024 · arxiv.org/abs/2404.01318

5. Hubert and Arabie, Comparing Partitions (ARI)
Journal of Classification, 1985

6. Project repository
github.com/Ananyacodes/jailbreaker-llm